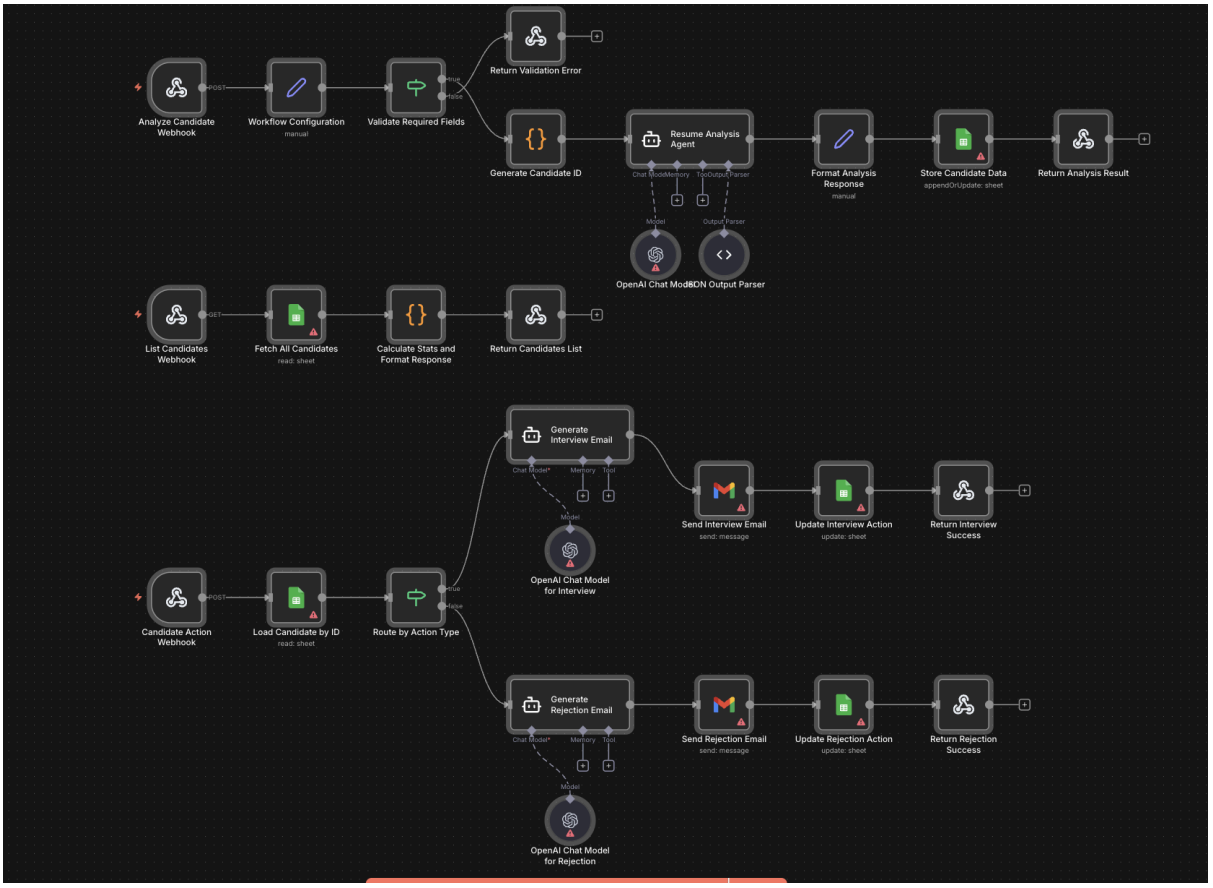
Agent Logic Overview (Recruit-AI)

The workflow is organized into **three logical agent-driven pipelines**, each triggered by a dedicated webhook and coordinated through routing, validation, and state persistence.



1. Candidate Analysis Agent (Resume Screening)

Trigger: `Analyze Candidate Webhook`

- Incoming payload is first **validated** to ensure all required fields (JD, resume, candidate info) are present.
- A **candidateId (UUID)** is generated early to uniquely track the candidate across the system.
- The **Resume Analysis Agent (LLM)** compares the resume against the job description and returns a **strictly structured JSON output** containing:
    - Fit score

    - Recommended action
    - Summary
    - Strengths and gaps
- An output parser ensures schema correctness before downstream use.
- Results are **normalized and formatted** for API response consistency.
- Full candidate data + AI output is **persisted in the datastore** for dashboard analytics.
- The workflow returns a clean, frontend-ready analysis response.

Purpose: Deterministic AI screening with explainable outputs and persistent state.

---

2. Candidate Listing & Metrics Agent (Dashboard)

Trigger: `List Candidates Webhook`

- Fetches candidates filtered by `clientUserId`.
- A lightweight **calculation agent (Function node)** derives aggregate metrics:
    - Total candidates reviewed
    - Interview recommendations
    - Pending actions
- Results are formatted into **stats + recent candidates**, optimized for dashboard rendering.

Purpose: Fast, query-based insights without invoking AI.

---

3. Candidate Action Agent (Interview / Rejection)

Trigger: `Candidate Action Webhook`

- Candidate record is **loaded by candidateId** to ensure action consistency.

- A **router node** directs execution based on action type:
    - `SCHEDULE_INTERVIEW`
    - `SEND_REJECTION`
- For each path:
    - A **specialized LLM email agent** generates context-aware, professional messaging.
    - Email is sent via Gmail.
    - Candidate record is updated with `lastAction` and `lastActionAt`.
- A confirmation response is returned to the frontend.

Purpose: Human-in-the-loop decision execution with AI-assisted communication.